Team Deliverable 4 Report

Brian Tramuel, Will Armentrout, Sam Suther, Isaac Vunzi, and Sana Ullah
Library Management System

A. Phase 2 Requirements

The scope of Phase 2 is unchanged from the plan established in Team Deliverable 2. Phase 2 builds directly on the authentication and catalog foundation delivered in Phase 1 and focuses on the borrow-and-return workflow, copy-count management, and associated security hardening. The requirements below are the same Phase 2 requirements specified in Deliverable 2, but have been expanded with detailed acceptance criteria and implementation notes.

Phase Overview

Phase	Focus	Included Requirements
Phase 1	MVP Auth and Catalog	FR-1 to FR-8, NFR-1 to NFR-4, NFR-9, NFR-10
Phase 2	Core Security, Borrow and Return	FR-9 to FR-16, NFR-5 to NFR-7
Phase 3	Admin Tools & Polish	FR-17 to FR-23, NFR-8, NFR-11, NFR-12

Functional Requirements

ID	Name	Detailed Description
FR-9	Book Availability Status	The system displays whether a book is available or checked out on both the catalog list and the book detail page. The availability badge updates in real time after a borrow or return action without requiring a full page reload.
FR-10	Borrow a Book	Authenticated users can borrow an available book by clicking the Borrow button on the detail page. The system inserts a borrowing_transactions record, decrements available_copies atomically, and returns the transaction details, including due date. Unauthenticated users are redirected to the login page.
FR-11	Loan Linked to Account	Each borrowed book is associated with the borrowing user's account via a foreign key in the borrowing_transactions table. A user cannot see or modify another user's loans.
FR-12	Due Date Storage	Each borrowing transaction stores a due_date calculated as 14 days from borrowed_at using a PostgreSQL DEFAULT expression. The due date is returned in the API response and displayed on the user's loan list.
FR-13	Return a Book	Authenticated users can return a book they have borrowed.

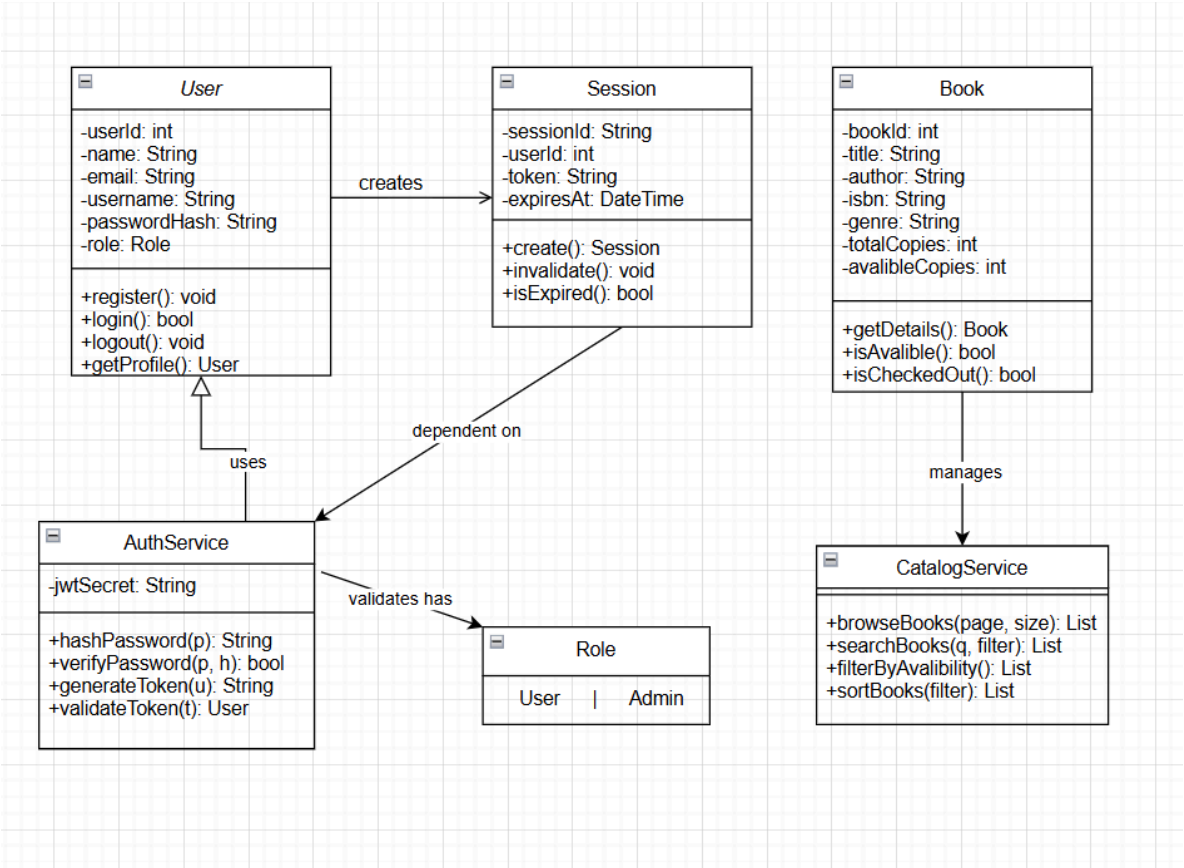
		The system atomically sets the transaction status to 'returned', records returned_at, and increments available_copies. The Borrow button becomes available again for other users.
FR-14	Prevent Borrowing Unavailable Books	The system checks available_copies before processing a borrow request. If copies is zero, the API returns HTTP 400 with the message 'No copies available'. The frontend disables the Borrow button when the book shows a Checked Out status.
FR-15	View Borrowed Books & Due Dates	Authenticated users can view a list of their active and past loans at GET /api/borrow/my. Each record shows book title, author, borrowed_at, due_date, and current status (active / returned / overdue)
FR-16	Borrowing Limit	The system enforces a maximum of three active loans per user. Attempting to borrow a fourth book returns HTTP 400 with the message 'Borrow limit of 3 reached'. The limit is checked server-side before the transaction begins.

Non-Functional Requirements

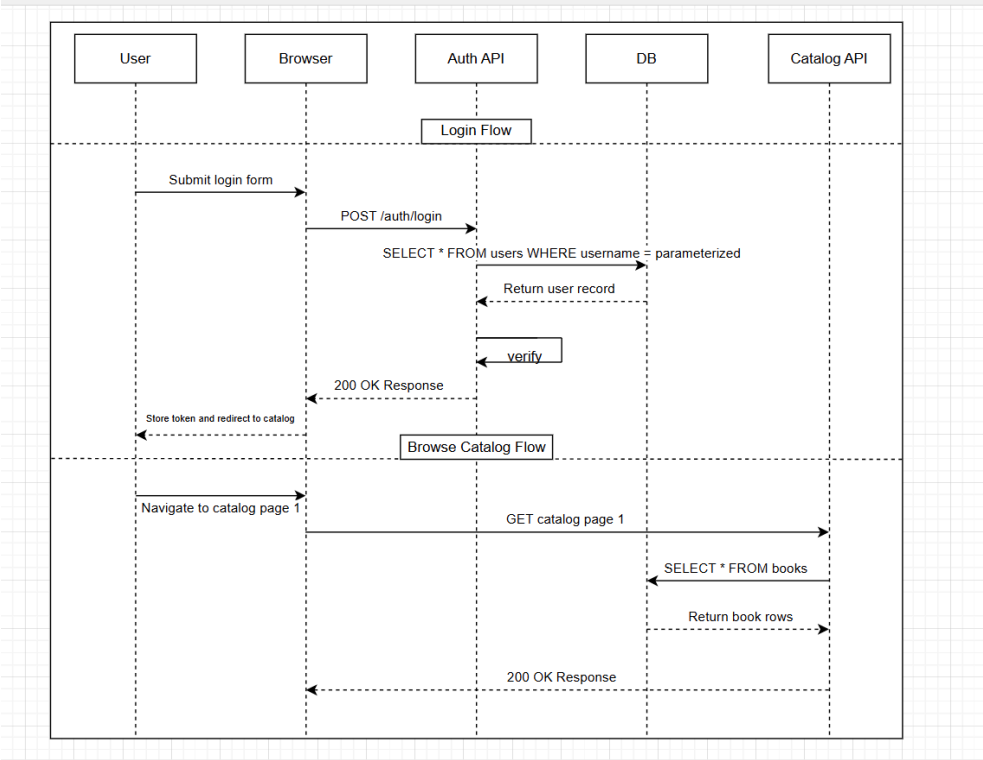
ID	Category	Requirement and Detail
NFR-5	Performance	95% of catalog and borrow/return API requests respond within 2 seconds under normal class-sized load (up to 25 concurrent users). Verified with manual load testing using concurrent curl requests.
NFR-6	Scalability	The system supports at least 25 concurrent users without failure or data corruption. The PostgreSQL connection pool is configured with a pool size appropriate for this load.
NFR-7	Atomicity	All borrow and return database operations are wrapped in BEGIN/COMMIT transactions. If any query within the transaction fails, a ROLLBACK is executed automatically, leaving the database in a consistent state with no partial updates to available_copies or transaction status.

B. UML Design For Phase 1

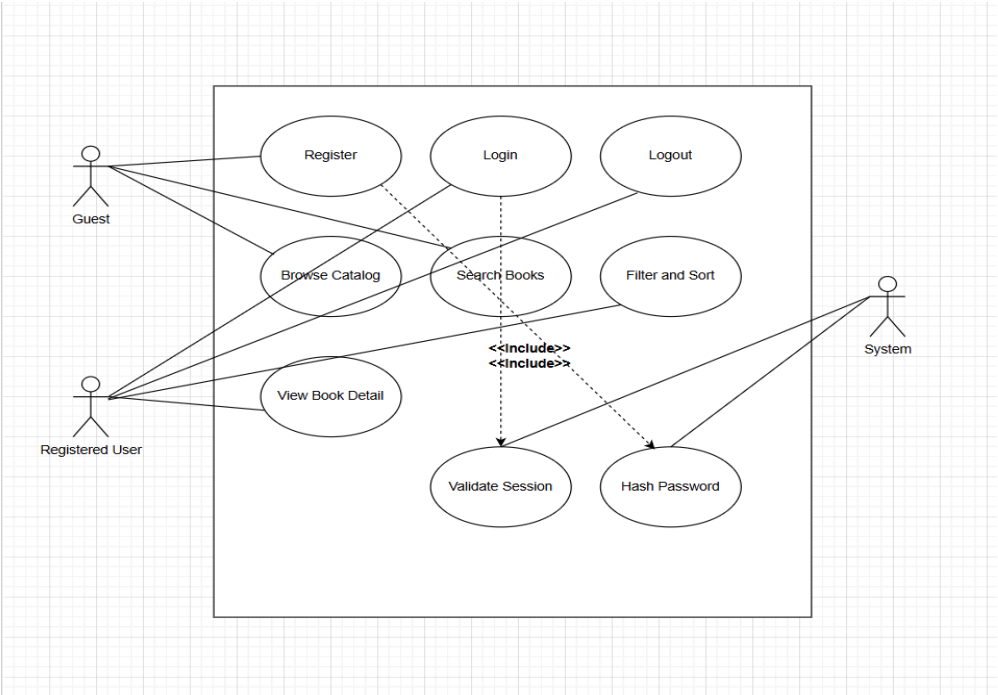
Class Diagram



Sequence Diagram



Use-Case Diagram



C. User Manual

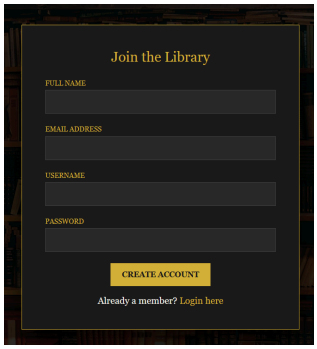
This manual explains how to access and use the Library Management System as it currently exists after Phase 1. The application is a web-based system with a Node.js/Express backend, a SQL database, and an HTML/CSS/JavaScript frontend styled with a dark theme.

Accessing the Application

Once the server is running, open your browser and go to *http://localhost:3000*. You will see the Library home page with a centered layout, a Dark Academia-themed design, and navigation links for Login, Register, and Search in the header.

Registering a New Account

1. Click **Register** in the navigation bar. The registration page will load.
2. Fill in your **Full Name, Email Address, Username, and Password**.
3. Your password must be at least **8** characters and include one uppercase letter, one lowercase letter, one digit, and one special character.
4. Click **Create Account**. On success, you are redirected to the login page. If the username or email is already taken, an error message will appear below the field.



Logging In and Out

Click Login in the navigation bar. Enter your username and password, then click Sign In. After a successful login, your session is stored, and you are redirected to the catalog. To log out, click the Logout link in the navigation bar; your session will be cleared immediately.

Searching and Browsing the Catalog

The Search page is accessible from the navigation bar at any time, whether or not you are logged in.

- **Search bar**: Type a book title or author name and press Enter or click Search. Results update on the page without a full reload.
- **Browse all**: Leave the search bar empty and submit to see all available books.
- **Book cards**: Each result card shows the title, author, genre, and availability status.
- **Book Detai**l: Click a book title or card to open its full detail page, which lists ISBN, genre, total copies, and available copies.

D. Compile/Run Instructions

The project uses Node.js and Express for the backend, PostgreSQL for the database, and plain HTML/CSS/JavaScript for the frontend. All source files are on the main branch of the GitHub repository.

Must Have:

- **Node.js**
- **Git**
- **PostgreSQL**

Step 1 - Clone Git

git clone https://github.com/ssuther6-ops/ITIS3300-Panthers.git cd ITIS3300-Panthers

Step 2 - Install Dependencies

The project dependencies are declared in package.json.
Install them with: **npm install**

Step 3 - Create and Configure the Database

Open a psql shell and create the database:
CREATE DATABASE olms;

Then run the schema script to create the tables:
psql -d olms -f src/database/schema.sql

Then load the seed data (sample books and admin user):
psql -d olms -f src/database/seed.sql

Step 4 - Set Environment Variables

Create a .env file in the project root and set the following:

Variable	Example Value

DB_Host	localhost
DB_Port	5432
DB_Name	olms
DB_User	postgres
DB_Password	Passw0rd
JWT_Secret	panthers3300
PORT	3000

The .env file isn't committed to the repo for security reasons; you have to create it yourself using the values above

Step 5 - Start the Server

node src/backend/server.js

You should see: OLMS server port 3000. Open *http://localhost:3000* in your browser to use the application.

Running Test

EndPoint	**Method**
/api/auth/register	POST
/api/auth/login	POST
/api/books	GET
/api/books?search=query	GET
/api/borrow	POST
/api/borrow/return	POST
/api/borrow/my	GET
/api/users	GET
/api/users/history	GET

E. Test Cases

#	Method	Input	Expected Output	Pass

				or Fail
1	POST /api/auth/register **(valid new user)**	{name:"Jane", email:"j@x.com", username:"jane1", password:"Pass@123" }	201 Created; user object returned; no password hash in response	Pass
2	POST /api/auth/register **(duplicate username)**	{ same username as test 1 }	400 Bad Request; 'Username or email already taken.'	Pass
3	POST /api/auth/register **(weak password)**	{ password: "abc" }	400 Bad Request; password complexity error message	Pass
4	POST /api/auth/login **(valid credentials)**	{ username:"jane1", password:"Pass@123" }	200 OK; JWT token returned in response body	Pass
5	POST /api/auth/login **(wrong password)**	{ username:"jane1", password:"wrong" }	401 Unauthorized; 'Invalid username or password.'	Pass
6	GET /api/books **(no auth, no filter)**	No body, no token	200 OK; JSON array of all active books sorted A-Z	Pass
7	GET /api/books?search =gatsby **(search filter)**	Query param search=gatsby	200 OK; only books with 'gatsby' in title or author returned	Pass
8	GET /api/books?available=true **(availability filter)**	Query param available=true	200 OK; only books with available_copies > 0 returned	Pass
9	POST /api/borrow **(borrow an available book)**	Valid JWT; { book_id: 1 }	201 Created; transaction row with due_date 14 days out	Pass

10	POST /api/borrow **(Borrow the same book twice)**	Same JWT and book_id as test 9	400 Bad Request; 'You already have this book borrowed.'	Pass
11	POST /api/borrow **(book unavailable)**	Valid JWT; book_id with available_copies=0	400 Bad Request; 'No copies available.'	Pass
12	POST /api/borrow **(exceed 3-book limit)**	Valid JWT; 4th borrow attempt	400 Bad Request; 'Borrow limit of 3 reached.'	Pass
13	POST /api/borrow/return **(return active loan)**	Valid JWT; { transaction_id: 1}	200 OK; 'Book returned successfully'; available_copies incremented	Pass
14	POST /api/borrow/return **(return someone else's loan)**	Different user JWT; same transaction_id	404 Not Found; 'Active transaction not found.'	Pass
15	GET /api/borrow/my **(view own loans)**	Valid JWT; no body	200 OK; an array of the users' borrow records with title, due_date, status	Pass
16	GET /api/users **(admin route, non-admin token)**	User-role JWT	403 Forbidden; 'Admin access only.'	Pass
17	GET /api/users **(admin route, admin token)**	Admin-role JWT	200 OK; array of all user accounts; no password_hash fields	Pass
18	GET /api/auth/profile **(valid token)**	Valid JWT	200 OK; authenticated user's profile without password hash	Pass
19	GET /api/auth/profile	No Authorization header	401 Unauthorized; 'Access denied. No token provided.'	Pass

	(no token)			

F. Code Inspection Feedback

Item	Inspection Feedback	Action Taken
7	In main.js, the password strength color logic resets strengthText.color to an empty string when metCount === 0 instead of using a named color. Hex color values are used throughout without named constants, making the code harder to read.	Added named color constants at the top of the strength-indicator block and replaced all inline hex values with those constants. The reset case now sets a consistent default color.
18	A potential directory mismatch was flagged in the comment in server.js, referring to 'backend/routes/db.js', but the actual path is 'backend/db.js', which could confuse new developers.	Updated all path comments in server.js and bookController.js to reflect the correct directory structure. Also ran a grep across all files to catch any other stale path references.
21	The MAX_BORROW_LIMIT constant in borrowController.js was noted as a 'magic number' with no explanation of why 3 was chosen.	Added a JSDoc comment above MAX_BORROW_LIMIT explaining the business rule and noting that the value can be changed in one place to affect the entire system.
16	'Some Falsy values used', the inspection noted that certain Boolean checks rely on JavaScript falsy evaluation rather than explicit null/undefined checks.	Reviewed all falsy checks in the auth middleware. Added explicit checks where ambiguity could cause bugs, particularly in edge cases where the Authorization header is malformed.
General Feedback	The password strength color logic should use named constants or comments so readers do not need a hex color picker to understand the intent.	Addressed under item 7 above. Colors are now named constants with comments explaining the strength level each represents.

G. Reflection

What has been accomplished:

Phase 2 is complete. The borrow and return workflows are fully implemented and tested. The borrowController.js module uses atomic PostgreSQL transactions to guarantee data consistency when borrowing or returning. The three-book borrowing limit, duplicate-loan prevention, and availability checks are all enforced server-side. The GET /api/borrow/my endpoint lets users view their full loan

history with due dates and statuses. Code inspection feedback from the Pintos team was reviewed, and the four flagged items were addressed in the codebase.

What Went Well:

- **Transactions:** The BEGIN/COMMIT/ROLLBACK pattern worked reliably. No data inconsistencies were observed during testing, even when simulating failures.
- Caught a real bug during testing: our original transaction code called pool.query('BEGIN') directly on the pool, which doesn't pin all queries to the same connection. We fixed it by checking out a single client with a pool. connect and running begin, commit, and rollback on that client.
- **Code inspection value:** The inspection session surfaced real readability issues that were quickly fixed.
- **Test coverage:** Writing explicit test cases before declaring Phase 2 complete caught the edge case of returning another user's loan, which was not originally in scope.
- **Payoff:** The borrow.js and borrowController.js stubs created in Phase 1 gave the team a clear head start and reduced integration friction.

Areas For Improvement:

- **Automated testing:** All test cases were run manually using Postman. Phase 3 should introduce a Jest or Mocha test suite so tests can be re-run automatically on every commit.
- **Feature branches:** Most Phase 2 commits still went directly to main. The team will adopt a stricter pull-request workflow in Phase 3.
- **Frontend integration:** The borrow/return API is complete, but the frontend UI for borrowing is still a placeholder. Connecting the frontend to the new endpoints is the first priority in Phase 3.

H. Contribution Table

Member Name	Contribution Description	Overall Contribution %	Note
Will	Team deliverable 4 report and many of the presentation slides.	**20**	
Brian	Backend API implementation: Caught and fixed the transaction atomicity bug, switched from pool. query to pool. connect for single-client transactions. End-to-end tested all 10 endpoints with curl, verified borrow/return with before/after DB checks. Wrote the backend test log.	**20**	
Isaac	Help Fix The Book Search Bar Problem and added more books in the Library Catalogue w/ Book Covers to make it more visually appealing.	**20**	

Sam	Helped update the main.js file to make passwords for the system more advanced and secure.	**20**	
Sana	Helped with refactoring for the navigation and book sections, and helped refactor the registration form, login structure, and user authentication.	**20**	